Chapter 6. Training and Optimization of Deep Networks

Mini-batch SGD, activations, weight initialization, learning rates, batch normalization; Taylor expansion and the descent lemma, SGD, momentum, Nesterov, AdaGrad, RMSProp, Adam

Contents

1 Full, Stochastic and Mini-Batch Gradients **2**

2 Activation Functions Revisited **2**

3 Weight Initialization **3**

3.1 Why the scale matters . . . 3

3.2 Zero and symmetric initialization . . . 3

3.3 Xavier and He initialization . . . 3

3.4 Variance of a sigmoid output (2025 midterm Q2.2) . . . 5

4 Learning-Rate Schedules **5**

5 Batch Normalization **5**

5.1 Training-time computation . . . 5

5.2 Test time . . . 6

6 Dropout **6**

7 Gradient Descent and Taylor's Theorem **6**

8 Smoothness and the Descent Lemma **7**

9 Stochastic Gradient Descent **7**

10 Momentum **8**

10.1 Heavy-ball momentum . . . 8

10.2 Nesterov momentum . . . 8

11 Adaptive Methods **9**

11.1 Exponential moving averages and bias correction (2024 midterm Q1) . . . 10

11.2 Special cases and invariances (2022 midterm Q1) . . . 10

11.3 Weight decay is not ℓ_2 regularization (2024 midterm Q7) . . . 10

1 Full, Stochastic and Mini-Batch Gradients

For $f(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} f_i(\boldsymbol{\theta})$, the full gradient costs N per-sample gradients. Choosing I uniformly, $\mathbf{g} = \nabla f_I$ is **unbiased**: $\mathbb{E}[\mathbf{g} \mid \boldsymbol{\theta}] = \frac{1}{N}\sum_i \nabla f_i = \nabla f$. A mini-batch averages b such gradients.

Theorem 6.1: mini-batch variance

If the b indices are drawn independently with replacement and a single draw has conditional covariance Σ, the mini-batch gradient has covariance Σ/b. Without replacement it is $\frac{N-b}{b(N-1)}\Sigma$ (with Σ normalized by N).

Proof. With replacement the b terms are iid with covariance Σ; the covariance of their average is $\frac{1}{b^2}\cdot b\Sigma$. Without replacement the pairwise covariances are $-\Sigma/(N-1)$, which gives the finite-population correction. ■

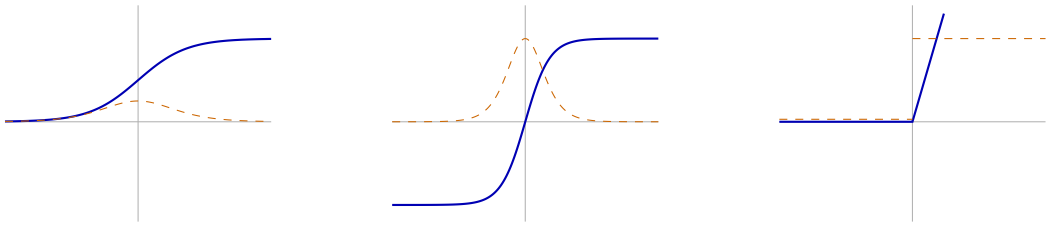
Example 6.2: unbiased but noisy

$f_i(w) = \frac{1}{2}(w - y_i)^2$ with $\mathbf{y} = (1, 3, 5)$ at $w = 0$: per-sample gradients $-1, -3, -5$, full gradient -3. A single random gradient averages to -3; a two-sample batch without replacement gives $-2, -3, -4$, again averaging to -3.

One *iteration* is one parameter update; one *epoch* is one pass through the data (1000 samples, batch 100: 10 iterations per epoch). Full batch: deterministic but expensive; single sample: cheap but noisy; mini-batch: a compromise.

2 Activation Functions Revisited

Sigmoid. Saturates: $\sigma' \le 1/4$ and $\sigma' \to 0$ for large $|z|$. Outputs are positive, so for the next layer $h = W\mathbf{z}$ (with $\mathbf{z}$ the positive sigmoid outputs) $\partial L/\partial W_{ij} = \delta_i z_j$ has the sign of δ_i for every j: all weights into a neuron move in the same direction in one step, producing zig-zag paths. **Tanh.** Zero-centered and odd, $\tanh' \in (0, 1]$, but still saturates near ± 1. **ReLU.** Cheap, derivative 1 on the positive side (no saturation there), but units with negative input get zero gradient (dead ReLU) and outputs are not centered. Leaky ReLU, ELU and maxout keep a nonzero slope or add flexibility.



Activation functions (solid) and their derivatives (dashed): the sigmoid and tanh *saturate, ReLU has derivative* 0 *or* 1.

3 Weight Initialization

3.1 Why the scale matters

`W = s * np.random.randn(Din, Dout)` draws iid $N(0, s^2)$ weights. For $z_j = \sum_{i=1}^{D_{\text{in}}} W_{ji} x_i$ with zero-mean independent weights of variance σ_W^2, independent of inputs with $\mathbb{E}x_i^2 = q$,

$$\mathbb{E}z_j = 0, \qquad \operatorname{Var} z_j = D_{\text{in}} \sigma_W^2 q$$

(cross terms vanish because $\mathbb{E}[W_{ji}W_{jk}] = 0$ for $i \neq k$). For six tanh layers of width 4096:

- $s = 0.01$: $D_{\text{in}} s^2 = 0.41$; activations shrink towards 0 layer by layer, and weight gradients $\propto$ inputs become tiny.
- $s = 0.05$: $D_{\text{in}} s^2 = 10.24$; pre-activations have standard deviation ≈ 3, tanh saturates (histograms pile up at ± 1), $\tanh' \approx 0$, and the backpropagated signal $\partial L/\partial \mathbf{z} = \sigma'(\cdot) W^\top \partial L/\partial \mathbf{h}$ is almost zero: no learning, although the loss is still large. A zero gradient caused by saturation is not a sign of optimality.

3.2 Zero and symmetric initialization

Theorem 6.3: zero initialization freezes a tanh MLP (2022 midterm Q5, 2025 midterm Q3)

For $f(\mathbf{x}) = A_L y_{L-1} + b_L$, $y_\ell = \tanh(A_\ell y_{\ell-1} + b_\ell)$, all parameters initialized to 0 and GD on $\frac{1}{2}\sum_i (f(X_i) - Y_i)^2$: $A_1, \ldots, A_L, b_1, \ldots, b_{L-1}$ stay 0 forever, and $b_L \to \bar{Y}$ if $0 < \eta < 2/N$.

Proof. Induction on the claim "all parameters except b_L are zero". Forward: $\tanh(0) = 0$ so every hidden output is 0 and $f = b_L$. Backward: $\partial/\partial A_L = \sum_i \delta_L^{(i)} y_{L-1}^\top = 0$; $\boldsymbol{\delta}_{L-1} = (A_L^\top \delta_L) \odot \tanh'(0) = 0$ since $A_L = 0$, and inductively all $\boldsymbol{\delta}_\ell = 0$, so $\partial A_\ell = \boldsymbol{\delta}_\ell y_{\ell-1}^\top = 0$, $\partial b_\ell = 0$. Hence GD does not move them. For b_L: $J = \frac{1}{2}\sum (b_L - Y_i)^2$, $b^{(t+1)} - \bar{Y} = (1 - \eta N)(b^{(t)} - \bar{Y})$. ■

Theorem 6.4: duplicate neurons stay duplicates (2021 midterm Q2)

For $f(x) = \sum_{j=1}^{p} u_j \sigma(a_j x + b_j)$ trained by Adam (or GD, momentum, RMSProp), if $(a_{p-1}, b_{p-1}, u_{p-1}) = (a_p, b_p, u_p)$ at initialization, they remain equal for all iterations.

Proof. $\partial f/\partial u_j = \sigma(a_j x + b_j)$, $\partial f/\partial a_j = u_j \sigma'(a_j x + b_j) x$, $\partial f/\partial b_j = u_j \sigma'(a_j x + b_j)$; the loss gradient multiplies these by $\partial \ell/\partial f$, which is shared by all neurons. So equal parameters give equal gradient coordinates. Induct on the statement "the paired coordinates of $\boldsymbol{\theta}^k$, m_1^k, m_2^k agree": it holds at $k = 0$ ($m_1^0 = m_2^0 = 0$), and every Adam operation (EMAs, bias corrections, elementwise division) acts coordinatewise. The optimizer state must be part of the induction hypothesis. ■

Hence identical neurons compute the same function forever: random initialization is needed to break symmetry.

3.3 Xavier and He initialization

For an approximately linear activation, preserving the second moment requires $D_{\text{in}} \sigma_W^2 = 1$, i.e. $\sigma_W = 1/\sqrt{D_{\text{in}}}$ (often called "Xavier"; Glorot's compromise $2/(D_{\text{in}} + D_{\text{out}})$ also balances the backward pass).

Theorem 6.5: He initialization (2022 midterm Q6)

Let $y^+ = A\mathbf{x}$, $\mathbf{x} = \mathrm{ReLU}(\mathbf{y})$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$, where $\mathbf{y}$ is independent of A, its coordinates have mean 0, variance 1, are uncorrelated, and $\mathbf{y} \overset{d}{=} -\mathbf{y}$. Then

(i) $\mathbb{E}x_j^2 = \frac{1}{2}$; (ii) $\mathbb{E}y^+ = 0$; (iii) $\mathbb{E}(y_i^+)^2 = \frac{n_{\text{in}}\sigma_A^2}{2}$;

(iv) $y_1^+, \ldots, y_{n_{\text{out}}}^+$ are uncorrelated; (v) $y^+ \overset{d}{=} -y^+$.

Thus $\sigma_A^2 = 2/n_{\text{in}}$ preserves the second moment through ReLU layers.

Proof. (i) $\mathrm{ReLU}(y)^2 = y^2\mathbf{1}\{y > 0\}$; symmetry makes the positive and negative halves of $\mathbb{E}y^2 = 1$ equal ($y = 0$ contributes nothing).

(ii) Independence and $\mathbb{E}A_{ij} = 0$.

(iii) $\mathbb{E}(y_i^+)^2 = \sum_{j,k} \mathbb{E}[A_{ij}A_{ik}]\mathbb{E}[x_j x_k] = \sum_j \sigma_A^2 \mathbb{E}x_j^2$.

(iv) For $i \neq k$ all $\mathbb{E}[A_{ij}A_{kl}] = 0$.

(v) $(A, \mathbf{y}) \overset{d}{=} (-A, \mathbf{y})$ since A is independent of $\mathbf{y}$ and symmetric; apply $F(A, \mathbf{y}) = A\,\mathrm{ReLU}(\mathbf{y})$. ■

Remark 6.6: refinements

Leaky ReLU with slope α: $\mathbb{E}\phi(y)^2 = \frac{1+\alpha^2}{2}$ and $\sigma_A^2 = \frac{2}{(1+\alpha^2)n_{\text{in}}}$. ReLU output is not centered: for $z \sim N(0, v)$, $\mathbb{E}\,\mathrm{ReLU}(z) = \sqrt{v/(2\pi)}$, $\mathbb{E}\,\mathrm{ReLU}(z)^2 = v/2$, so He preserves the second moment, not the variance. Backward, $\partial\ell/\partial\mathbf{x} = A^\top \partial\ell/\partial\mathbf{y}^+$ has variance $n_{\text{out}}\sigma_A^2$: forward and backward preservation ask for n_{in} versus n_{out}.

Theorem 6.7: sigmoid layer (2021 midterm Q4)

For $\mathbf{y} = \sigma(A\mathbf{x})$ with $\sigma(t) \approx \frac{1}{2} + \frac{t}{4}$, $\sigma' \approx \frac{1}{4}$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$ independent of $\mathbf{x}$, and x_j uncorrelated with mean $\frac{1}{2}$ and variance 1: $\mathbb{E}y_i = \frac{1}{2}$, $\mathrm{Var}\, y_i = \frac{5n_{\text{in}}\sigma_A^2}{64}$. If the upstream gradients $\partial\ell/\partial y_i$ have mean 0, variance 1, are uncorrelated and independent of A, then $\partial\ell/\partial x_j \approx \frac{1}{4}\sum_i A_{ij}\partial\ell/\partial y_i$ has mean 0 and variance $\frac{n_{\text{out}}\sigma_A^2}{16}$.

Proof. $\mathrm{Var}(A_{ij}x_j) = \mathbb{E}A_{ij}^2\mathbb{E}x_j^2 = \sigma_A^2(1 + \frac{1}{4})$ and the terms are uncorrelated; multiply by $\frac{1}{16}$. The backward computation is identical with $\mathbb{E}(\partial\ell/\partial y_i)^2 = 1$. ■

Theorem 6.8: convolution layer (2024 midterm Q5)

For a 3×3, padding-1 convolution $Y_{cij} = \sum_{p,q,r} w_{cpqr}\tilde{X}_{p,i+q-2,j+r-2} + b_c$ with zero-mean unit-variance uncorrelated inputs, $w \overset{\text{iid}}{\sim} N(0, \sigma_w^2)$, $b \overset{\text{iid}}{\sim} N(0, \sigma_b^2)$ independent: $\mathbb{E}Y_{cij} = 0$ everywhere, and $\mathbb{E}Y_{cij}^2 = \sigma_w^2 \cdot \#\{\text{in-range inputs}\} + \sigma_b^2$, i.e. $9C_{\text{in}}\sigma_w^2 + \sigma_b^2$ in the interior, $6C_{\text{in}}\sigma_w^2 + \sigma_b^2$ on edges and $4C_{\text{in}}\sigma_w^2 + \sigma_b^2$ at corners.

Proof. Expand the square: weight–weight terms leave $\sigma_w^2\mathbb{E}\tilde{X}_a^2$ on the diagonal, weight–bias terms vanish, and $\mathbb{E}b_c^2 = \sigma_b^2$. Padded entries are zero. ■

3.4 Variance of a sigmoid output (2025 midterm Q2.2)

For smooth f and X with mean μ, Taylor's theorem $f(X) = f(\mu) + f'(\mu)(X-\mu) + \frac{1}{2}f''(\xi)(X-\mu)^2$ gives $\operatorname{Var} f(X) = f'(\mu)^2 \operatorname{Var} X + \text{Error}$, the error collecting terms with f''. For $X \sim N(0,1)$ and $Y = \sigma(wX+b)$, $\operatorname{Var} Y \approx w^2\sigma'(b)^2$: small weights or large $|b|$ (saturation) give small output variance. An exact bound also holds: if f is L-Lipschitz, $\operatorname{Var} f(X) \le \mathbb{E}(f(X)-f(\mu))^2 \le L^2 \operatorname{Var} X$, so $\operatorname{Var} Y \le w^2/16$.

4 Learning-Rate Schedules

The learning rate controls the update size: too large diverges, too small is slow, and a large early rate followed by a smaller late rate refines the solution and reduces gradient noise.

- Step: multiply by 0.1 at fixed epochs (ResNet: 30, 60, 90), giving $\eta_0, 0.1\eta_0, 0.01\eta_0, 0.001\eta_0$.
- Cosine: $\eta_t = \frac{\eta_0}{2}[1+\cos(\pi t/T)]$; linear: $\eta_t = \eta_0(1-t/T)$; inverse square root: $\eta_t = \eta_0/\sqrt{t}$.
- Linear warmup: increase from 0 to $\eta_{\max}$ over the first iterations (e.g. 5000), then decay.

5 Batch Normalization

5.1 Training-time computation

For a mini-batch $X \in \mathbb{R}^{B\times D}$ (rows: samples, columns: features),

$$\mu_j = \frac{1}{B}\sum_i X_{ij}, \quad v_j = \frac{1}{B}\sum_i (X_{ij}-\mu_j)^2, \quad \hat{X}_{ij} = \frac{X_{ij}-\mu_j}{\sqrt{v_j+\epsilon}}, \quad Y_{ij} = \gamma_j \hat{X}_{ij} + \beta_j.$$

γ, β are learned, so the network can choose the output scale and center (with $\gamma_j = \sqrt{v_j+\epsilon}$, $\beta_j = \mu_j$ it recovers the input for that batch).

Example 6.9: two features

$X = \begin{pmatrix} 1 & 10 \\ 3 & 14 \end{pmatrix}$: $\mu = (2, 12)$, $v = (1, 4)$, $\hat{X} = \begin{pmatrix} -1 & -1 \\ 1 & 1 \end{pmatrix}$ (ϵ ignored). With $\gamma = (2, 3)$, $\beta = (-1, 5)$, $Y = \begin{pmatrix} -3 & 2 \\ 1 & 8 \end{pmatrix}$.

Theorem 6.10: invariances

With $\epsilon = 0$:
(i) each column of $\hat{X}$ has mean 0 and mean square 1;
(ii) for $z = \mathbf{w}^\top \mathbf{x} + b$ the BN output does not depend on b and is invariant under $\mathbf{w} \mapsto c\mathbf{w}$, $c > 0$;
(iii) consequently $\langle \nabla_{\mathbf{w}} L, \mathbf{w} \rangle = 0$.

Proof. (ii) $z - \mu = \mathbf{w}^\top(\mathbf{x} - \bar{\mathbf{x}})$ removes b; scaling $\mathbf{w}$ scales both $z-\mu$ and $\sqrt{v}$ by c.
(iii) $h(c) = L(c\mathbf{w})$ is constant, so $0 = h'(1) = \nabla L(\mathbf{w})^\top \mathbf{w}$. ■

5.2 Test time

At inference the batch may have size one, and the output must not depend on the other samples. During training BN keeps running averages

$$\mu^{\text{run}} \leftarrow m\,\mu^{\text{run}} + (1-m)\mu_{\text{batch}}, \qquad (\sigma^2)^{\text{run}} \leftarrow m\,(\sigma^2)^{\text{run}} + (1-m)\sigma^2_{\text{batch}}$$

(an exponential moving average, as in Adam; PyTorch's `momentum` argument is $1-m$), and at test time uses

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j^{\text{run}}}{\sqrt{(\sigma_j^2)^{\text{run}} + \epsilon}}, \qquad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

With a batch of one and batch statistics, $\mu = x$ and $v = 0$ would give $\hat{x} = 0$: all information lost. In code: `model.train()` versus `model.eval()`.

Theorem 6.11: fusion at inference

At test time BN is the affine map $z \mapsto az + c$ with $a_j = \gamma_j/\sqrt{v_j^{\text{run}} + \epsilon}$, $c_j = \beta_j - a_j\mu_j^{\text{run}}$, so $\text{BN}(W\mathbf{x} + \mathbf{b}) = \text{diag}(\mathbf{a})W\mathbf{x} + (\text{diag}(\mathbf{a})\mathbf{b} + \mathbf{c})$ can be folded into the preceding layer.

Remark 6.12: why BN helps

Reported benefits: easier optimization, better gradient flow, larger learning rates, less sensitivity to initialization, mild regularization through batch noise. The original "internal covariate shift" explanation has been challenged; a smoother loss landscape is an alternative view. In training mode BN couples samples in the backward pass: with $g_i = \partial L/\partial y_i$ and $r = \sqrt{v + \epsilon}$, $\partial L/\partial x_i = \frac{\gamma}{r}(g_i - \bar{g} - \hat{x}_i\,\overline{g\hat{x}})$.

6 Dropout

Inverted dropout multiplies each unit by $m_i/(1-p)$ with $m_i \sim \text{Bernoulli}(1-p)$ during training (so $\mathbb{E}$ = identity) and does nothing at evaluation. It prevents co-adaptation and acts like training an ensemble of sub-networks (AlexNet applies it to the FC layers).

Theorem 6.13: dropout and activations commute iff positively homogeneous (2021 midterm Q3)

With the same mask, $\phi(H \odot \mathbf{z}) = H \odot \phi(\mathbf{z})$ for all masks iff $\phi(0) = 0$ and $\phi(cz) = c\phi(z)$ for $c = 1/(1-p)$. This holds for ReLU and LeakyReLU, but fails for the sigmoid.

Proof. Coordinatewise, a dropped unit gives $\phi(0)$ versus 0, a kept unit $\phi(cz)$ versus $c\phi(z)$. For $c > 0$, ReLU and LeakyReLU preserve sign cases and scale linearly. For the sigmoid with $p = 0.4$ and $z = 0$: $\sigma(0)/0.6 \approx 0.833 \neq \sigma(0) = 0.5$. ■

7 Gradient Descent and Taylor's Theorem

We minimize $f : \mathbb{R}^d \to \mathbb{R}$; the iterate $\mathbf{x}_t$ is the whole parameter vector. GD is $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\nabla f(\mathbf{x}_t)$, and $-\nabla f$ is the steepest-descent direction since $\nabla f^\top \mathbf{v} \geq -\|\nabla f\|$ for unit $\mathbf{v}$ (Cauchy–Schwarz).

Theorem 6.14: Taylor along a segment

With $\mathbf{d} = \mathbf{y} - \mathbf{x}$ and $h(t) = f(\mathbf{x} + t\mathbf{d})$,

$$f(\mathbf{y}) - f(\mathbf{x}) = \int_0^1 \nabla f(\mathbf{x}+t\mathbf{d})^\top \mathbf{d}\, dt, \qquad f(\mathbf{y}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{d} + \int_0^1 (1-t)\, \mathbf{d}^\top \nabla^2 f(\mathbf{x}+t\mathbf{d})\mathbf{d}\, dt.$$

Proof. The first is $h(1) - h(0) = \int_0^1 h'$. For the second, $h(1) = h(0) + h'(0) + \int_0^1 \int_0^t h''(s)\, ds\, dt$ and exchanging the order over $0 \le s \le t \le 1$ gives $\int_0^1 (1-s) h''(s)\, ds$. ■

8 Smoothness and the Descent Lemma

Definition 6.15: L-smoothness

f is L-smooth if $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \le L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$. For $f \in C^2$ this is equivalent to $-L\mathbf{I} \preceq \nabla^2 f \preceq L\mathbf{I}$ (all Hessian eigenvalues in $[-L, L]$).

Theorem 6.16: descent lemma

If f is L-smooth, $f(\mathbf{y}) \le f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2}\|\mathbf{y} - \mathbf{x}\|^2$.

Proof. Subtract $\nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 \nabla f(\mathbf{x})^\top \mathbf{d}\, dt$ from the first Taylor formula, then use Cauchy–Schwarz and smoothness:

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 (\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x}))^\top \mathbf{d}\, dt \le \int_0^1 Lt\|\mathbf{d}\|^2 dt = \frac{L}{2}\|\mathbf{d}\|^2.$$

■

Theorem 6.17: GD on smooth functions

GD satisfies $f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta(1 - \frac{L\eta}{2})\|\nabla f(\mathbf{x}_t)\|^2$, so it decreases f when $0 < \eta < 2/L$. With $\eta = 1/L$ and $f^\star = \inf f > -\infty$,

$$\min_{0 \le t < T} \|\nabla f(\mathbf{x}_t)\|^2 \le \frac{1}{T} \sum_{t<T} \|\nabla f(\mathbf{x}_t)\|^2 \le \frac{2L\,(f(\mathbf{x}_0) - f^\star)}{T}.$$

Proof. Put $\mathbf{y} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$ in the descent lemma. With $\eta = 1/L$ the coefficient is $1/(2L)$; summing over t telescopes the left side to $f(\mathbf{x}_0) - f(\mathbf{x}_T) \le f(\mathbf{x}_0) - f^\star$. ■

This is a stationarity guarantee: for nonconvex f a small gradient may sit at a saddle point or a local minimum. For $f(x) = x^2/2$ ($L = 1$), $x_{t+1} = (1 - \eta)x_t$ converges iff $0 < \eta < 2$; for $f = \frac{1}{2}\mathbf{x}^\top H \mathbf{x}$ each eigen-direction contracts by $1 - \eta\lambda_i$, so a small eigenvalue (poor conditioning) means slow progress while $\eta < 2/\lambda_{\max}$ is required for stability.

9 Stochastic Gradient Descent

SGD uses $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \mathbf{g}_t$ with $\mathbb{E}_t \mathbf{g}_t = \nabla f(\mathbf{x}_t)$, where $\mathbb{E}_t$ conditions on the history up to $\mathbf{x}_t$.

Theorem 6.18: second moment is not variance

$\mathbb{E}_t\|\mathbf{g}_t\|^2 = \|\nabla f(\mathbf{x}_t)\|^2 + \mathbb{E}_t\|\mathbf{g}_t - \nabla f(\mathbf{x}_t)\|^2$.

Proof. Expand $\mathbf{g}_t = \nabla f + (\mathbf{g}_t - \nabla f)$; the cross term has conditional mean zero. ■

Theorem 6.19: $T^{-1/2}$ rate for SGD

If f is L-smooth, $\mathbb{E}_t\|\mathbf{g}_t\|^2 \le G$ and $\Delta = f(\mathbf{x}_0) - f^\star$, then for a constant step

$$\frac{1}{T}\sum_{t<T}\mathbb{E}\|\nabla f(\mathbf{x}_t)\|^2 \le \frac{\Delta}{\eta T} + \frac{L\eta G}{2}, \qquad \text{and with } \eta = \sqrt{\tfrac{2\Delta}{LGT}}: \quad \le \sqrt{\frac{2L\Delta G}{T}}.$$

Proof. Descent lemma: $f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\nabla f(\mathbf{x}_t)^\top\mathbf{g}_t + \frac{L\eta^2}{2}\|\mathbf{g}_t\|^2$. Conditional expectation gives $\mathbb{E}_t f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\|\nabla f(\mathbf{x}_t)\|^2 + \frac{L\eta^2}{2}G$. Take total expectation (tower property), sum over $t < T$ (telescoping), use $\mathbb{E}f(\mathbf{x}_T) \ge f^\star$ and divide by ηT. Finally $\frac{a}{\eta} + b\eta$ is minimized at $\eta = \sqrt{a/b}$ with value $2\sqrt{ab}$. ■

The noise term forces $\eta \sim T^{-1/2}$, slower than GD's T^{-1}. Since mini-batches of size B divide the noise variance by B, larger batches reduce G and permit larger steps.

10 Momentum

10.1 Heavy-ball momentum

Definition 6.20: momentum

$\mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{x}_t)$, $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}$, $\mathbf{v}_0 = \mathbf{0}$, $0 \le \beta < 1$.

Unrolling, $\mathbf{v}_{t+1} = -\eta\sum_{k=0}^{t}\beta^{t-k}\nabla f(\mathbf{x}_k)$: an exponentially weighted sum of past gradients (a velocity). For a constant gradient $\mathbf{g}$, $\|\mathbf{v}_t\| \to \eta\|\mathbf{g}\|/(1-\beta)$, an effective step $1/(1-\beta)$ times larger ($10\times$ for $\beta = 0.9$). Persistent directions accumulate; oscillating directions cancel, which reduces zig-zagging in narrow valleys. Physically the gradient acts as a force and $1-\beta$ as friction: in the buffer form $\mathbf{m}_{t+1} = \mathbf{g}_t + \beta\mathbf{m}_t$ one has $\mathbf{m}_{t+1} - \mathbf{m}_t = \mathbf{g}_t - (1-\beta)\mathbf{m}_t$.

Theorem 6.21: momentum on a quadratic

For $f(x) = \frac{h}{2}x^2$, $x_{t+1} = (1+\beta-\eta h)x_t - \beta x_{t-1}$; the iteration converges iff both roots of $r^2 - (1+\beta-\eta h)r + \beta = 0$ lie in the open unit disk, which for $0 \le \beta < 1$ holds iff $0 < \eta h < 2(1+\beta)$. Complex roots (modulus $\sqrt{\beta}$) mean damped oscillation.

Proof. Substitute $\mathbf{v}_{t+1} = x_{t+1} - x_t$ and $\mathbf{v}_t = x_t - x_{t-1}$ into the velocity update. For the root condition use the Jury/Schur criterion for $r^2 + a_1 r + a_0$: $|a_0| < 1$, $1 + a_1 + a_0 > 0$, $1 - a_1 + a_0 > 0$, which here read $\beta < 1$, $\eta h > 0$, $\eta h < 2(1+\beta)$. ■

10.2 Nesterov momentum

Nesterov evaluates the gradient at the look-ahead point:

$$\mathbf{y}_t = \mathbf{x}_t + \beta\mathbf{v}_t, \qquad \mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{y}_t), \qquad \mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}.$$

Example 6.22: a numerical illustration

$f = x^2/2$, $x_0 = 1$, $\eta = 0.1$, $\beta = 0.9$. Both methods give $v_1 = -0.1$, $x_1 = 0.9$. Momentum: $v_2 = 0.9(-0.1) - 0.1(0.9) = -0.18$, $x_2 = 0.72$. Nesterov: $y_1 = 0.81$, $v_2 = -0.09 - 0.081 = -0.171$, $x_2 = 0.729$: it measures the slope where it is about to go and brakes earlier (not necessarily closer to the optimum at every step).

Theorem 6.23: two forms of Nesterov (2021 midterm Q6)

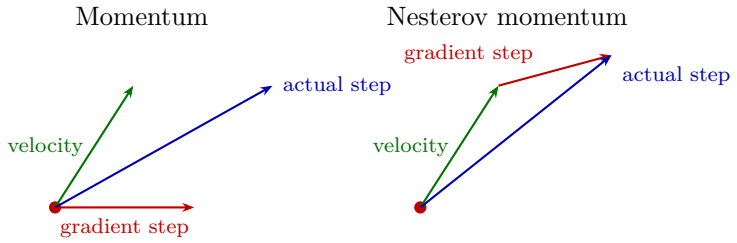
Form I: $\psi^{k+1} = \theta^k - \alpha g^k$, $\theta^{k+1} = \psi^{k+1} + \beta(\psi^{k+1} - \psi^k)$, $\psi^0 = \theta^0$. Form II (PyTorch `nesterov=True`): $m^{k+1} = g^k + \beta m^k$, $v^{k+1} = \beta m^{k+1} + g^k$, $\theta^{k+1} = \theta^k - \alpha v^{k+1}$, $m^0 = 0$. Here g^k is evaluated at θ^k. For the same θ^0 and $g^0, g^1, \ldots$ both produce the same $\theta^1, \theta^2, \ldots$

Proof. *Both satisfy one recursion.* Form I with $\psi^{k+1} = \theta^k - \alpha g^k$ and $\psi^k = \theta^{k-1} - \alpha g^{k-1}$ gives, for $k \geq 1$,

$$\theta^{k+1} = \theta^k - \alpha g^k + \beta(\theta^k - \theta^{k-1}) - \alpha\beta(g^k - g^{k-1}).$$

Form II: $\theta^{k+1} = \theta^k - \alpha g^k - \beta(\alpha g^k + \alpha\beta m^k)$, and from the previous step $\alpha\beta m^k = \alpha v^k - \alpha g^{k-1} = \theta^{k-1} - \theta^k - \alpha g^{k-1}$; substituting gives the same recursion. Both start with $\theta^1 = \theta^0 - \alpha(1+\beta)g^0$. A second-order recursion with the same two initial values has the same solution (induction). *Alternative.* Define $\psi^k := \theta^k + \alpha\beta m^k$ in Form II. Then $\psi^0 = \theta^0$, $\psi^{k+1} = \theta^k - \alpha g^k$ and $\psi^{k+1} - \psi^k = -\alpha m^{k+1}$, from which Form I's second equation follows. ■

Interpretation: ψ is the actual position (often written x_t), θ the look-ahead point $x_t + \beta v_t$ at which gradients are taken and which PyTorch stores as the parameter; m is the velocity of ψ scaled by $-1/\alpha$. Both converge to the same limit since $\theta^k - \psi^k = \beta(\psi^k - \psi^{k-1}) \to 0$.



Momentum adds the gradient step taken at the current point to the velocity; Nesterov evaluates the gradient after first moving along the velocity.

11 Adaptive Methods

All operations below are coordinatewise.

Definition 6.24: AdaGrad, RMSProp, Adam

AdaGrad: $\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$, $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\, \mathbf{g}_t \oslash (\sqrt{\mathbf{r}_t} + \epsilon)$.
RMSProp: $\mathbf{r}_t = \rho \mathbf{r}_{t-1} + (1-\rho)\mathbf{g}_t \odot \mathbf{g}_t$, same update.
Adam: $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$, $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t \odot \mathbf{g}_t$, $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$, $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$, $\mathbf{x}_t = \mathbf{x}_{t-1} - \eta\, \hat{\mathbf{m}}_t \oslash (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$. Typical $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$.

Remark 6.25: what each piece does

AdaGrad's accumulator grows forever, so its effective rate $\eta/\sqrt{r_t}$ shrinks (for a constant gradient g the step is $\eta/\sqrt{t}$). RMSProp forgets old gradients; for constant $g = 2$, $\rho = 0.9$, $\eta = 0.1$, $r_1 = 0.4$, $r_2 = 0.76$, steps $0.316, 0.229$, tending to 0.1. Adam = momentum's numerator (direction) + RMSProp's denominator $\sqrt{\mathbf{v}}$ (per-coordinate scale) + bias correction. (Nesterov plus Adam is NAdam.)

11.1 Exponential moving averages and bias correction (2024 midterm Q1)

Theorem 6.26: EMA of iid gradients

If $g_0, g_1, \dots$ are iid with mean μ and variance s^2, and $m_{k+1} = \beta m_k + (1-\beta)g_k$, $m_0 = 0$, then

$$\mathbb{E}m_k = (1-\beta^k)\mu, \qquad \operatorname{Var} m_k = s^2 \frac{1-\beta}{1+\beta}(1-\beta^{2k}),$$

with limits μ and $s^2\frac{1-\beta}{1+\beta}$. The corrected $\hat{m}_k = m_k/(1-\beta^k)$ is unbiased with variance $s^2\frac{(1-\beta)(1+\beta^k)}{(1+\beta)(1-\beta^k)}$ (equal to s^2 at $k=1$).

Proof. m_k depends only on $g_0, \dots, g_{k-1}$, so it is independent of g_k. Mean by induction: $\beta(1-\beta^k)\mu + (1-\beta)\mu = (1-\beta^{k+1})\mu$. Variance: $\operatorname{Var} m_{k+1} = \beta^2 \operatorname{Var} m_k + (1-\beta)^2 s^2$; writing $(1-\beta)^2 = \frac{1-\beta}{1+\beta}(1-\beta^2)$, the induction step is $\beta^2(1-\beta^{2k}) + (1-\beta^2) = 1-\beta^{2k+2}$. Correction: divide by $(1-\beta^k)^2$ and use $1-\beta^{2k} = (1-\beta^k)(1+\beta^k)$. ■

Without correction Adam's first step is $\frac{1-\beta_1}{\sqrt{1-\beta_2}}\operatorname{sign}(g_1)\eta \approx 3.16\,\eta$; with correction $\hat{m}_1 = g_1$, $\hat{v}_1 = g_1^2$ and the step is exactly $\eta \operatorname{sign}(g_1)$. The correction removes the zero-initialization bias at the price of a larger early variance.

11.2 Special cases and invariances (2022 midterm Q1)

Theorem 6.27: Adam contains RMSProp and signSGD

For Adam without bias correction: $\beta_1 = 0$ gives RMSProp with $\rho = \beta_2$; $\beta_1 = \beta_2 = 0$, $\epsilon = 0$ give signSGD $\theta_{k+1} = \theta_k - \eta \operatorname{sign}(g_k)$ (if no coordinate of g_k is 0). With $\epsilon = 0$, Adam's iterates are unchanged when the loss is multiplied by $c > 0$, while SGD's step scales by c.

Proof. $\beta_1 = 0$ makes $m_{k+1} = g_k$. With $\beta_2 = 0$ also, $\sqrt{v_{k+1}} = |g_k|$. Scaling $g \mapsto cg$ scales m by c and $\sqrt{v}$ by c. ■

11.3 Weight decay is not ℓ_2 regularization (2024 midterm Q7)

For plain SGD, ℓ_2 regularization $\theta - \eta(g + \lambda\theta) = (1-\eta\lambda)\theta - \eta g$ is the same as weight decay. With momentum it is not:

Theorem 6.28: no matching exists

SGD with momentum $v_{k+1} = g_k + \beta v_k$, $\theta_{k+1} = \theta_k - \alpha v_{k+1}$ and $\alpha, \beta > 0$. Weight decay: $\theta_{k+1} = \theta_k - \alpha v_{k+1} - \mu\theta_k$. ℓ_2: $v_{k+1} = (g_k + \lambda\theta_k) + \beta v_k$. No choice of $\mu, \lambda > 0$ makes the two methods equivalent.

Proof. Take $g_k \equiv 0$, $v_0 = 0$, $\theta_0 \neq 0$. Weight decay: $v_k = 0$ and $\theta_k = (1-\mu)^k\theta_0$. ℓ_2: $v_1 = \lambda\theta_0$, $\theta_1 = (1-\alpha\lambda)\theta_0$, $v_2 = \lambda(1-\alpha\lambda+\beta)\theta_0$, $\theta_2 = [(1-\alpha\lambda)^2 - \alpha\beta\lambda]\theta_0$. Matching θ_1 forces $\mu = \alpha\lambda$, and then matching θ_2 forces $\alpha\beta\lambda = 0$, a contradiction. (Indeed the ℓ_2 sequence is geometric for no ratio.) ■

In Adam the ℓ_2 gradient $\lambda\theta$ is divided by its own RMS, so every weight moves by about η regardless of its size ($\theta_1 = \theta_0 - \eta\,\mathrm{sign}(\theta_0)$ when $g \equiv 0$), losing proportional shrinkage; AdamW applies the decay $(1-\eta\lambda)\theta$ outside the adaptive scaling.